

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

Факультет **Прикладной Информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №6.2

По дисциплине «Проектирование и реализация баз данных»

Тема: Работа с БД в СУБД MongoDB

Выполнил Авдиенко Д.А.; К3240.

Преподаватель Говорова М.М.

Дата 29.05.2026

Санкт-Петербург 2026

СОДЕРЖАНИЕ

Ход работы.....	4
Задание 2.1.1.....	4
Задание 2.2.1.....	4
Задание 2.2.2.....	7
Задание 2.2.3.....	7
Задание 2.2.4.....	8
Задание 2.3.1.....	8
Задание 2.3.2.....	9
Задание 2.3.3.....	9
Задание 2.3.4.....	9
Задание 3.1.1.....	10
Задание 3.1.2.....	11
Задание 3.2.1.....	11
Задание 3.2.2.....	12
Задание 3.2.3.....	12
Задание 3.3.1.....	12
Задание 3.3.2.....	13
Задание 3.3.3.....	14
Задание 3.3.4.....	14
Задание 3.3.5.....	14
Задание 3.3.6.....	16
Задание 3.3.7.....	17
Задание 3.4.1.....	18
Задание 4.1.1.....	19
Задание 4.2.1.....	20

Задание 4.3.1.....	20
Задание 4.4.1.....	21

Ход работы

Задание 2.1.1

Была создана БД learn и заполнена коллекция соответствующими командами

```
[learn> use learn
already on db learn
[learn> show dbs
admin    40.00 KiB
config   92.00 KiB
local    40.00 KiB
learn> db.unicorns.insertOne({name: 'Horny', loves: ['carrot', 'papaya'], weight:
600, gender: 'm', vampires: 63})
| db.unicorns.insertOne({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450
, gender: 'f', vampires: 43})
| db.unicorns.insertOne({name: 'Unicrom', loves: ['energon', 'redbull'], weight:
984, gender: 'm', vampires: 182})
| db.unicorns.insertOne({name: 'Roooooodles', loves: ['apple'], weight: 575, gen
```

Далее был добавлен еще один объект через переменную

```
[learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, ge
nder: 'm', vampires: 165}
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
[learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a180909eb05d9b19f64ed64')
}
[learn> db.unicorns.find()
```

После, я проверил содержимое коллекции с помощью метода find и получил вывод 12-ти (всех) объектов

База данных learn была создана. Коллекция unicorns была заполнена документами. Документ Dunx был добавлен вторым способом: сначала был создан объект document, затем он был вставлен в коллекцию. Содержимое коллекции проверено методом find().

Задание 2.2.1

Далее были выведены списки самцов (весь) и самок (3)

```

[learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a180909eb05d9b19f64ed64'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed59'),

```

```

[learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5a'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5e'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed61'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]

```

После были найдены самки, которые любят мокровь (все и первая через FindOne)

```

[learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5a'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5d'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed63'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
[learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a1808c9eb05d9b19f64ed5a'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> █

```

Затем, первая через limit

```

[learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5a'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]

```

Задание 2.2.2

Далее был модифицирован запрос для исключения из вывода информации о предпочтениях и поле (запрос для самцов)

```
[learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
[
  {
    _id: ObjectId('6a180909eb05d9b19f64ed64'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed59'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5f'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed62'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed60'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
]
```

Задание 2.2.3

Требуется вывести список единорогов в обратном порядке добавления

```
[learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a180909eb05d9b19f64ed64'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed63'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed62'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed61'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
]
```

Задание 2.2.4

Здесь требуется вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор

```
learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

Задание 2.3.1

Требуется вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора

Это было выполнено командой

```
learn> db.unicorns.find(
|   {gender: 'f', weight: {$gte: 500, $lte: 700}},
|   {_id: 0}
| )
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Задание 2.3.2

Требуется вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора

```
learn> db.unicorns.find(
|   {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
|   {_id: 0}
| )
[
|   {
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 39
|   }
| ]
```

Задание 2.3.3

Требуется найти всех единорогов, не имеющих ключ vampires

```
[learn> db.unicorns.find({vampires: {$exists: false}})
[
|   {
|     _id: ObjectId('6a1808c9eb05d9b19f64ed63'),
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
```

Задание 2.3.4

Нужно было вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {name: 1, loves: {$slice: 1}, _id: 0}
| ).sort({name: 1})
[
|   { name: 'Dunx', loves: [ 'grape' ] },
|   { name: 'Horny', loves: [ 'carrot' ] },
|   { name: 'Kenny', loves: [ 'grape' ] },
|   { name: 'Pilot', loves: [ 'apple' ] },
|   { name: 'Raleigh', loves: [ 'apple' ] },
|   { name: 'Roooooodles', loves: [ 'apple' ] },
|   { name: 'Unicrom', loves: [ 'energon' ] }
| ]
```

Задание 3.1.1

Требуется создать коллекцию с документами, сформировать запросы

```
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    populatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    populatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    populatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a181222eb05d9b19f64ed65'),
    '1': ObjectId('6a181222eb05d9b19f64ed66'),
    '2': ObjectId('6a181222eb05d9b19f64ed67')
  }
}
```

Независимые мэры

```
learn> db.towns.find(
|   {"mayor.party": "I"},
|   {name: 1, mayor: 1, _id: 0}
[ ] )
[
|   { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
| ]
```

Беспартийные мэры

```
learn> db.towns.find(
|   {"mayor.party": {$exists: false}},
|   {name: 1, mayor: 1, _id: 0}
[ ] )
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

Задание 3.1.2

Требуется

1. Сформировать функцию для вывода списка самцов единорогов.
2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> fn = function() { return this.gender == "m"; }
[Function: fn]
learn> var cursor = db.unicorns.find(fn).sort({name: 1}).limit(2)
```

3. Вывести результат, используя forEach.

```
learn> cursor.forEach(function(obj) {
|   print(obj.name)
| })
[MongoInvalidArgumentError: Query filter must be a plain object or ObjectId]
learn> var cursor = db.unicorns.find({gender: "m"}).sort({name: 1}).limit(2)

learn> cursor.forEach(function(obj) {
|   print(obj.name)
| })
Dunx
Horny
```

4. Содержание коллекции единорогов unicorns:

Был создан курсор для выборки самцов единорогов. Результат был отсортирован по имени, ограничен первыми двумя документами и выведен с помощью forEach().

Задание 3.2.1

Требуется Вывести количество самок единорогов весом от полутонны до 600 кг.

```
[learn> db.unicorns.find({gender: "f", weight: {$gte: 500, $lte: 600}}).count()
(node:70178) [MONGODB DRIVER] Warning: cursor.count is deprecated and will
be removed in the next major version, please use `collection.estimatedDocumentCount` or `collection.countDocuments` instead
(Use `node --trace-warnings ...` to show where the warning was created)
2
```

Задание 3.2.2

Требуется Вывести список предпочтений.

```
[learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Задание 3.2.3

Требуется Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([
  |   {$group: {_id: "$gender", count: {$sum: 1}}}
[ ] ])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

Задание 3.3.1

Требуется Выполнить команду:

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'],
| weight: 340, gender: 'm'})
[ ]
TypeError: db.unicorns.save is not a function
learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340,
gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a1813a3eb05d9b19f64ed68')
}
```

Команда была заменена на современный аналог. В коллекцию unicorns был добавлен документ Barney. Результат проверен с помощью find().

```

    weight: 840,
    gender: 'f'
  },
  {
    _id: ObjectId('6a180909eb05d9b19f64ed64'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1813a3eb05d9b19f64ed68'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]

```

Задание 3.3.2

Требуется

1. Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вапмира.

```

learn> db.unicorns.updateOne(
|   {name: 'Ayna'},
|   {$set: {weight: 800, vampires: 51}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

2. Проверить содержимое коллекции unicorns.

```

    vampires: 80
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5e'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5f'),
    name: 'Kenny',

```

Задание 3.3.3

Требуется

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.

```
learn> db.unicorns.updateOne(
|   {name: 'Raleigh'},
|   {$set: {loves: ['redbull']}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2. Проверить содержимое коллекции unicorns.

```
gender: 'm',
vampires: 39
},
{
  _id: ObjectId('6a1808c9eb05d9b19f64ed60'),
  name: 'Raleigh',
  loves: [ 'redbull' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
{
  _id: ObjectId('6a1808c9eb05d9b19f64ed61'),
  name: 'Leia',
```

Задание 3.3.4

Требуется

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции unicorns.

Задание 3.3.5

Требуется

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции towns.

```

learn> db.unicorns.updateMany(
|   {gender: 'm'},
|   {$inc: {vampires: 5}}
[] )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
[learn> db.unicorns.find({gender: 'm'})
[
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed59'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5b'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5c'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5f'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],

```

```

learn> db.towns.updateOne(
|   {name: "Portland"},
|   {$unset: {"mayor.party": 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: "Portland"})
[
  {
    _id: ObjectId('6a181222eb05d9b19f64ed67'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]

```

Задание 3.3.6

Требуется

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.
2. Проверить содержимое коллекции unicorns.

```

learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed62'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn>

```

Задание 3.3.7

Требуется

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a1808c9eb05d9b19f64ed5a'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> █
```

Задание 3.4.1

Требуется создать коллекцию с документами из методички

1. Удалите документы с беспартийными мэрами.
2. Проверьте содержание коллекции.

```
learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a181550eb05d9b19f64ed6a'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a181550eb05d9b19f64ed6b'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

3. Очистите коллекцию.

4. Просмотрите список доступных коллекций.

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
```

Задание 4.1.1

Требуется

1. Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
learn> db.habitats.insertMany([
|   {
|     _id: "forest",
|     full_name: "Forest",
|     description: "Forest habitat zone"
|   },
|   {
|     _id: "mountain",
|     full_name: "Mountain",
|     description: "Mountain habitat zone"
|   },
|   {
|     _id: "lake",
|     full_name: "Lake",
|     description: "Lake habitat zone"
|   }
| ])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountain', '2': 'lake' }
}
```

2. Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
3. Проверьте содержание коллекции единорогов.
4. Содержание коллекции единорогов unicorns:

```

    upsertedCount: 0
  }
learn> db.unicorns.updateOne(
|   {name: "Leia"},
|   {$set: {habitat: {$ref: "habitats", $id: "lake"}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {name: 1, habitat: 1, _id: 0}
| )
[
  { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') },
  { name: 'Leia', habitat: DBRef('habitats', 'lake') }
]

```

Задание 4.2.1

Требуется

1. Проверьте, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique.
2. Содержание коллекции единорогов unicorns:

```

learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {name: 1, habitat: 1, _id: 0}
| )
[
  { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') },
  { name: 'Leia', habitat: DBRef('habitats', 'lake') }
]
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]

```

Задание 4.3.1

Требуется

1. Получите информацию о всех индексах коллекции unicorns .
2. Удалите все индексы, кроме индекса для идентификатора.
3. Попытайтесь удалить индекс для идентификатора.

```
[learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
[learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
[learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
[learn> db.unicorns.dropIndex("name_1")
{ nIndexWas: 2, ok: 1 }
[learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
[learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> █
```

Вывод:

Был просмотрен список индексов коллекции unicorns. Созданный ранее индекс name_1 был удалён. При попытке удалить индекс _id_ MongoDB вывела ошибку, так как индекс по идентификатору является обязательным и не может быть удалён.

Задание 4.4.1

Требуется

1. Создайте объемную коллекцию numbers, задействовав курсор:
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}

```
learn> for(i = 0; i < 100000; i++){
|   db.numbers.insertOne({value: i})
| }
{
  acknowledged: true,
  insertedId: ObjectId('6a181696eb05d9b19f66740b')
}
```

2. Выберите последних четыре документа.

```
[learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a181696eb05d9b19f66740b'), value: 99999 },
  { _id: ObjectId('6a181696eb05d9b19f66740a'), value: 99998 },
  { _id: ObjectId('6a181696eb05d9b19f667409'), value: 99997 },
  { _id: ObjectId('6a181696eb05d9b19f667408'), value: 99996 }
]
```

3. Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)

`executionTimeMillis: 29,`

4. Создайте индекс для ключа `value`.
5. Получите информацию о всех индексах коллекции `numbers`.
6. Выполните запрос 2.

```
[learn> db.numbers.createIndex({value: 1})
value_1
[learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
[learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a181696eb05d9b19f66740b'), value: 99999 },
  { _id: ObjectId('6a181696eb05d9b19f66740a'), value: 99998 },
  { _id: ObjectId('6a181696eb05d9b19f667409'), value: 99997 },
  { _id: ObjectId('6a181696eb05d9b19f667408'), value: 99996 }
]
```

7. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
8. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

`executionTimeMillis: 29 > executionTimeMillis: 0.` Следовательно, мы можем сделать вывод, что с индексом запрос является более эффективным, так как он выполняется быстрее и требует меньше полного просмотра документов коллекции

Вывод

В ходе лабораторной работы были выполнены основные операции работы с MongoDB: создание коллекций, вставка документов, выборка данных, работа с

массивами и вложенными объектами, использование курсоров, агрегирование, изменение и удаление данных. Также были созданы ссылки между документами, настроены индексы и выполнено сравнение времени выполнения запроса с индексом и без индекса.